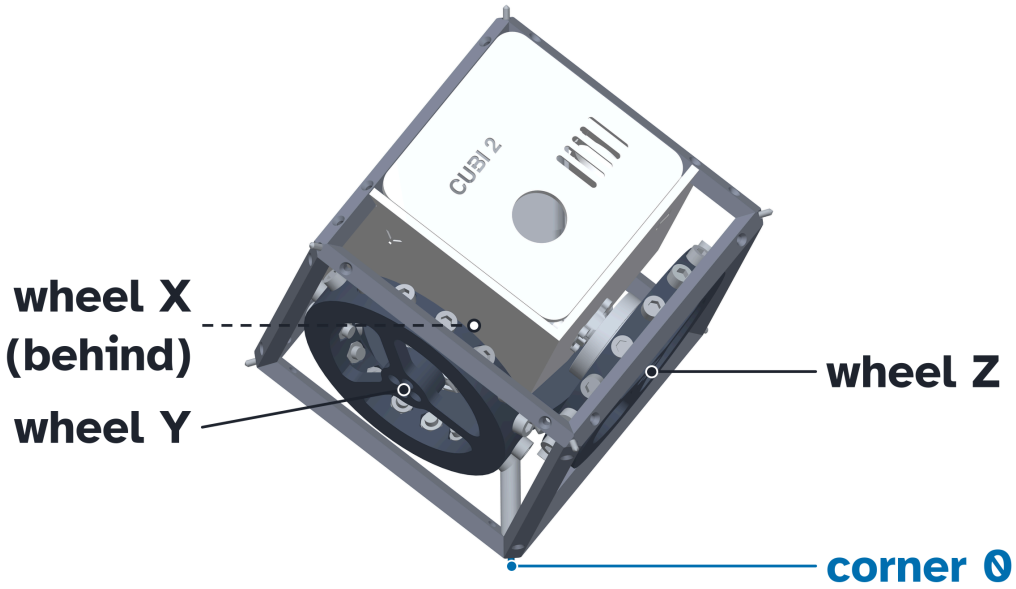
Start-up Guide

Bring Cubi to Life

The programs, the settings, the first balance - for a high-school student, on a laptop with Windows, macOS or Linux, with VS Code.



Designed by Athanasia Nikolova

All data come from tests and many real builds, not only from theory. v1.2 · November 2026

Teacher only



One thing per page

One action, one big picture. Short sentences, much white space.



Color is never the only clue

Every color also has a number, a letter or a name next to it. The part colors stay distinct with color-vision deficiency (Okabe-Ito palette).



Clear print

Instructions at 14 pt, everything else at least 12 pt, in Atkinson Hyperlegible. The text has a contrast of at least 4.5 : 1 to the page (WCAG 2.x, AA).



For a screen reader

The PDF is tagged (PDF/UA-1): headings, lists and reading order are marked, and every picture has a text alternative that is read aloud.



Print it right

In color, at 100 % - not 'fit to page'.



Pace

One page, then the action, then the next page. For a student who tires quickly: one lesson per session. The check at the end of each lesson is a good place to stop.

Safety first



The battery

6 cells, 22.2 V - a lot of energy. Only your teacher charges it, with a balance charger, in a LiPo bag.



Firsts with your teacher

The first power-up, the first spin and the first balance: always with your teacher.



Spinning wheels

Hands, hair and cables away from the cage.



Emergency stop

Pull the battery plug. 'stop', or Stop in the app, switches the wheels off too - type it WITHOUT touching the cube, then catch it.



One power at a time

USB cable or battery - never both at the same time.



A safe place

Balancing: on a rubber mat, on a low table, nothing fragile around. The cube can fall.



Smell or smoke

A smell, smoke, a hot part: battery out at once, tell your teacher.

Before you start

How to use this guide



One step at a time

A page holds a few steps. Do one step, then read the next.



8 lessons

30-45 minutes each. Tired? Stop after any step - the cube waits.



A check at the end

Each lesson ends with a check: tick the squares.



An error?

First copy it into a search engine or ask an AI assistant - that is what programmers do. Then your teacher. Battery, smoke, heat: your teacher at once.



Words and names

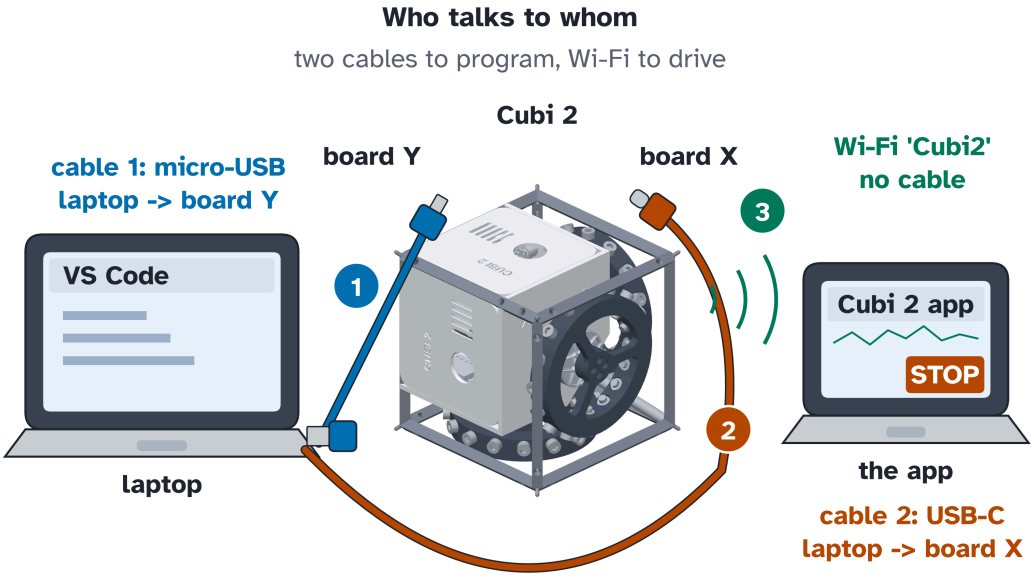
New words are explained at the end: 'Words'. Nothing here needs color: every wire has a name, every board a letter, every pole a number.

Dark boxes show the laptop's screen. Lines starting with '>' are what you TYPE (bold); the other lines are the answer you should see:

Terminal - Windows: PowerShell · macOS and Linux: Terminal — type, then Enter

```
› what you type
    the answer you should see
```

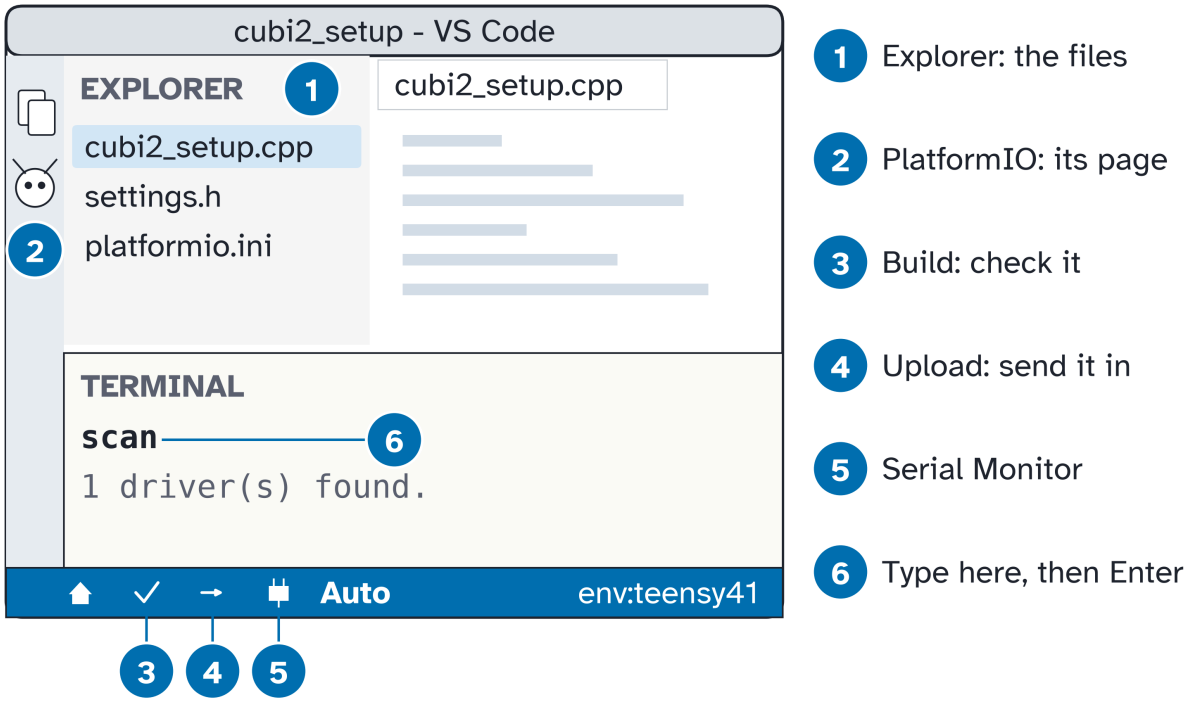
What you need



- A laptop with Windows 10 or 11, macOS, or Linux (Rocky Linux 9 or another RHEL-like), internet, and the right to install programs (on a school laptop ask the IT teacher; on Linux the 'sudo' password).
- A micro-USB DATA cable (Teensy) and a USB-C DATA cable (Wi-Fi board). Some cables only charge - they will not work.
- The file cubi2_firmware.zip: the 3 programs and INSTRUCTIONS.html (the same steps, short, with moving pictures). The Cubi 2 app comes from GitHub (lesson 1, 'Node.js and the app'). The laptop needs Wi-Fi.
- The charged battery, a multimeter, a rubber mat, your teacher nearby, a friend for lesson 6.

The tools, in short

VS Code - the 6 places you use

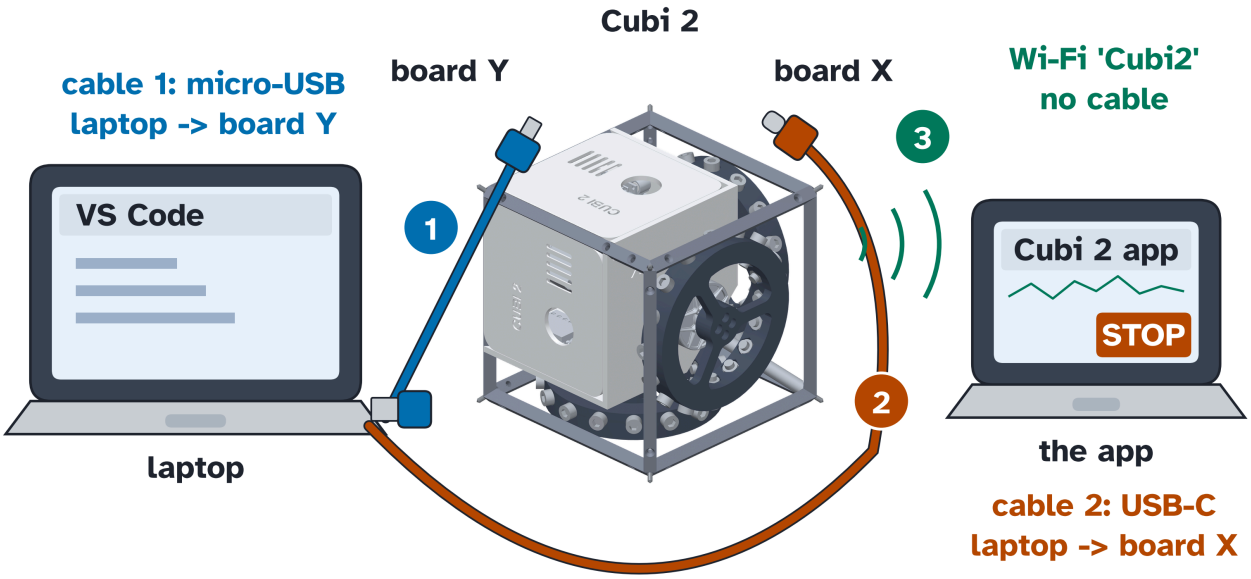


- **VS Code**
  the editor, for Windows, macOS and Linux: you read and write the programs here - the same as programmers use
- **PlatformIO**
  an extension of VS Code: Build (turns the program into machine code), Upload (sends it into the small computer), Serial Monitor (talk to it)
- **platformio.ini**
  a file in each program folder: the board, its settings and the libraries are written there - nothing to choose by hand
- **The app**
  the Cubi 2 app, in the browser of the laptop: talks to the cube over its Wi-Fi, without a cable
- **Node.js**
  runs the app's small server on the laptop - one line in a terminal
- **Multimeter**
  measures volts and checks that wires are joined (beep)

- **Not needed**
  the driver maker's program (Tinymovr Studio): the Teensy sets up the drivers itself.

Who talks to whom

two cables to program, Wi-Fi to drive

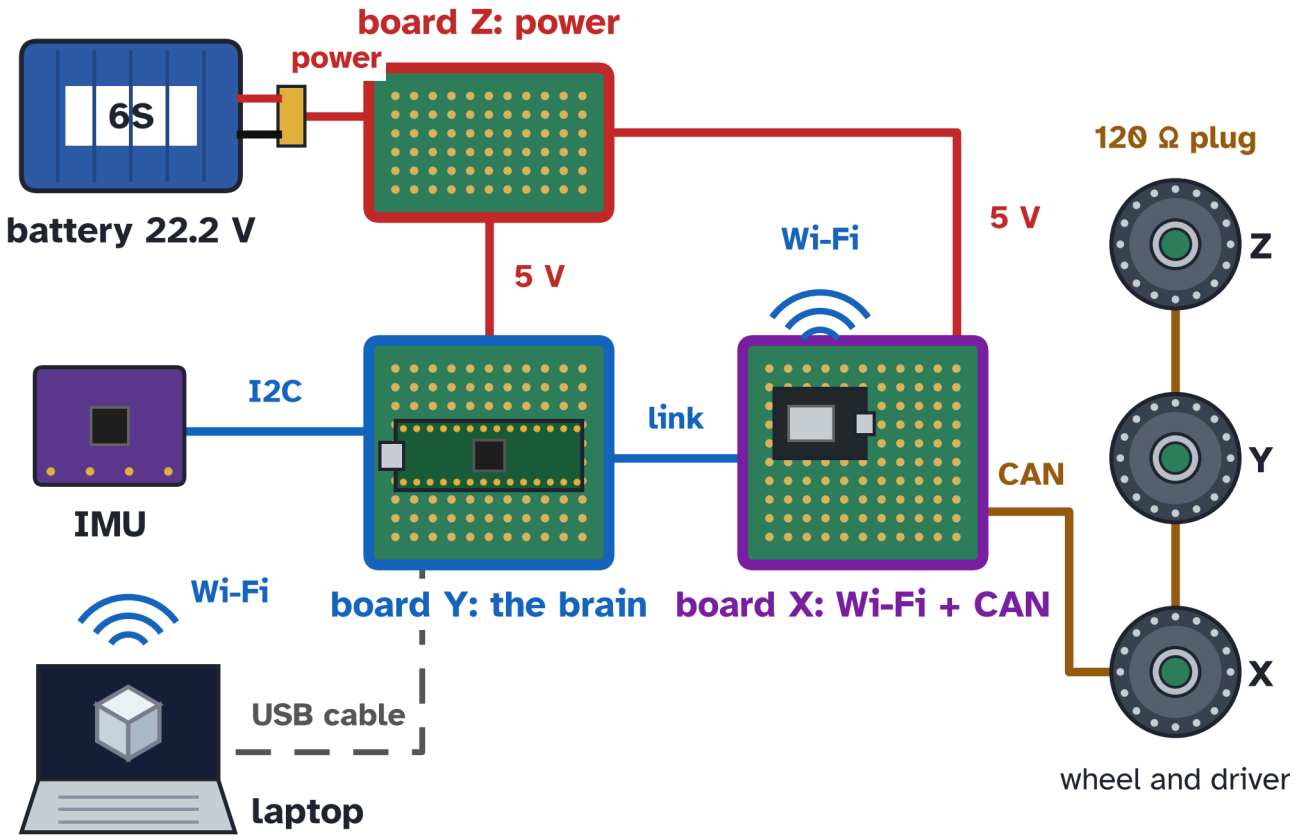


1 and 2: only to send in a program, battery OUT. 3: everything else.

- Cable 1 and cable 2: only to send a program in, and for the Serial Monitor.
- Battery OUT while a cable is in: USB cable or battery, never both.
- Wi-Fi: the app on the laptop talks to the cube - no cable.
- At the start of each lesson the same picture shows the one you use.

How it works

Three programs on two small computers.



- Teensy 4.1 on board Y is the brain. 250 times a second it reads the IMU (which way is up) and tells each motor driver how hard to push its wheel.
- The wheel pushes back on the cube. That is what holds it up.
- The Wi-Fi board on board X makes the network 'Cubi2'. The app on the laptop joins it.

Lesson 1 · The laptop

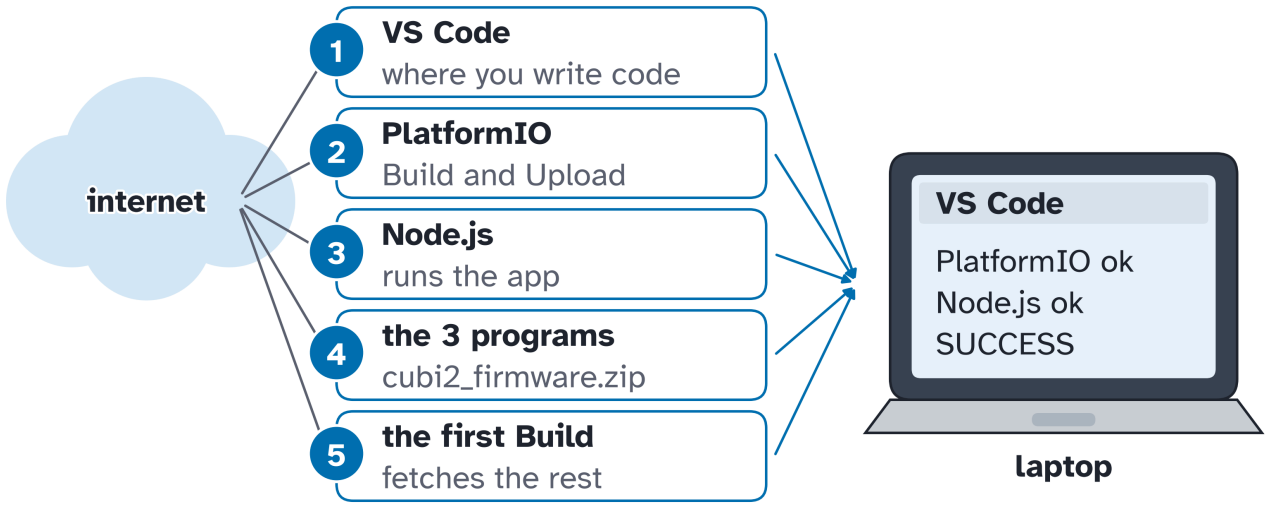
Install VS Code, PlatformIO, Node.js and the app. Once per laptop.

About 50 minutes. You can stop after any step. You will learn:

- what an editor, an extension and a build are
- how a program folder is made
- how to find help for an error yourself

What goes into the laptop

once per laptop - then it is ready for every cube



Lesson 1 does it step by step. Nothing to plug in yet.

Install VS Code

- Browser: the address below, then Download - the button for your system.

Browser - the address bar — type, then Enter

```
› code.visualstudio.com
```

- Windows: run the installer: accept, Next, Install. 'Do you want to allow … changes?' - Yes.
- macOS: open the .zip, drag 'Visual Studio Code' into Applications. First start: 'downloaded from the internet - open?' - Open.
- Linux (Rocky Linux 9, RHEL, AlmaLinux): download the .rpm, then in a terminal the lines below - the last one makes sure Python 3 is there (PlatformIO needs it).
- PATH: not needed here - VS Code and PlatformIO find their own tools. See 'PATH' at the end.

Terminal - Linux (Rocky Linux, RHEL-like) — type, then Enter

```
› cd ~/Downloads
› sudo dnf install ./code-*.rpm
    ...  Installed: code-...  Complete!
› sudo dnf install python3
    Package python3-3.9... is already installed.  Complete!
```

Windows only, once per laptop: let Windows use long file names - the tools of the Wi-Fi board have some, and without this their install stops ('exceeds Windows length limits'). Right-click Start > Terminal (Admin) > Yes; the line must start with PS C:\WINDOWS\system32. Then restart the laptop.

Windows - Terminal (Admin): right-click Start > Terminal (Admin) — type, then Enter

```
› New-ItemProperty -Path "HKLM:\SYSTEM\
CurrentControlSet\Control\FileSystem" -Name
"LongPathsEnabled" -Value 1 -PropertyType DWORD -Force
  LongPathsEnabled : 1
  PSPath           : Microsoft.PowerShell.Core\Registry::
HKEY_LOCAL_MACHINE\...
```

'Requested registry access is not allowed' = not an Admin terminal. No Admin right at all? Then, in a normal terminal, this line puts PlatformIO in a short folder instead - then close VS Code and open it again:

Windows - Terminal (PowerShell), a normal one — type, then Enter

```
› [Environment]::SetEnvironmentVariable(
"PLATFORMIO_CORE_DIR", "C:\pio", "User")
```

Add PlatformIO

- Left bar: Extensions (the four small squares). Type in the search box, then Install:

VS Code - Extensions (the four small squares): the search box — type, then Enter

```
› PlatformIO IDE
    PlatformIO IDE    [ Install ]
```

- It brings the C/C++ extension (and on Windows and macOS its own Python). Wait: the bottom right shows the work, a few minutes.
- 'Reload' when asked. A new icon appears in the left bar: PlatformIO.
- Linux: it says Python is missing? The last line of 'Install VS Code', then Reload.

The programs

- Unzip cubi2_firmware.zip into Documents - Windows: right-click > Extract All; macOS: double-click; Linux: right-click > Extract Here. You get Documents/firmware with cubi2_setup, cubi2_brain and cubi2_radio.
- Each folder is one program: a .cpp file (the program), some .h files (its parts, in C++) and platformio.ini. One VS Code window = one program; to change program: File > Open Folder.
- VS Code: File > Open Folder (macOS: File > Open) > Documents/firmware/cubi2_setup. 'Do you trust the authors?' - Yes. The blue bottom bar of PlatformIO appears.
- One folder open at a time. VS Code shows several in its left bar? File > Close Workspace, then Open Folder again.

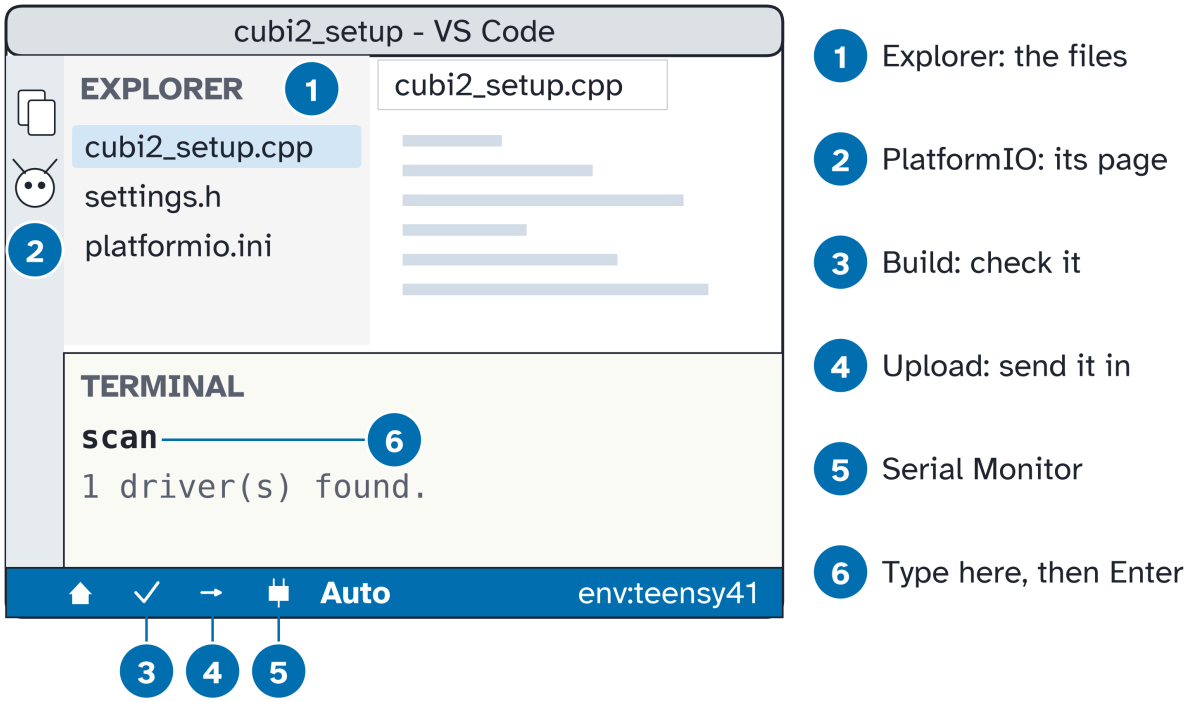
The first Build

- Bottom bar: the check mark (Build). The first time it downloads the Teensy tools and the IMU library - several minutes, internet needed.
- The end: SUCCESS. FAILED: copy the first line with 'error' into a search engine, or ask an AI assistant 'what does this error mean and how do I fix it?'. Still stuck: your teacher.
- Do the same for cubi2_brain and cubi2_radio (the Wi-Fi board's tools are the biggest).

VS Code - the terminal at the bottom, after the check mark (Build)

```
  Processing teensy41 (platform: teensy; board: teensy41;
framework: arduino)
  ...
  ========================= [SUCCESS] Took 41.20 seconds =======
==================
```

VS Code - the 6 places you use



The USB rights - macOS and Linux only

- macOS: the first time you plug a board in, it may ask 'Allow accessory to connect?' - Allow.
- Linux, once per laptop, in a terminal (lets PlatformIO reach the boards); then plug the board in again:

Terminal - Linux (Rocky Linux, RHEL-like) — type, then Enter

```
› curl -fsSL https://raw.githubusercontent.com/
platformio/platformio-core/develop/platformio/assets/
system/99-platformio-udev.rules | sudo tee /etc/udev/
rules.d/99-platformio-udev.rules
› sudo udevadm control --reload-rules && sudo udevadm
trigger
```

The first line is long. A \ at the end of a row means 'the line goes on': type it all as one line, without the \, then Enter once.

Node.js and the app

- Node.js runs the app on the laptop. Browser: the address below, then the LTS button for your system; install it (accept, Next, Install). Linux: the line below.
- The app: browser github.com/nasianikolova/Cubi-2 > the green Code button > Download ZIP. Unzip it, then copy its folder Code/cubi2app into Documents.

Browser - the address bar — type, then Enter

```
› nodejs.org
```

Terminal - Linux (Rocky Linux, RHEL-like) — type, then Enter

```
› sudo dnf module install nodejs:20
    ...  Complete!
```

- Start the app: a terminal, the lines below. node -v must show v18 or newer. Leave this terminal open while you use the app; Ctrl+C stops it.

Terminal - Windows: PowerShell · macOS and Linux: Terminal — type, then Enter

```
› node -v
    v20...
› cd ~/Documents/cubi2app
› node tools/serve.mjs
    Cubi2 app on http://localhost:8000
    the cube at http://192.168.4.1 - in the app's Cubi card
put: /cube
```

- Browser: the address below - the app opens. Without the cube it runs the Simulation.

Browser - the address bar — type, then Enter

```
› localhost:8000/html/index.html
```



Check - tick each line. All ticked: a good place to stop.

- ☐ VS Code opens; the PlatformIO icon is in the left bar.
- ☐ cubi2_setup, cubi2_brain, cubi2_radio: each Build ends with SUCCESS.
- ☐ Linux: the USB rights are in (see 'The USB rights').
- ☐ The app opens at localhost:8000/html/index.html (see 'Node.js and the app').

Next: lesson 2, power checks.

Lesson 2 · Power checks

Measure before the first power-up. With your teacher.

About 30 minutes. You can stop after any step. You will learn:

- how to check a circuit with the multimeter
- the one rule for the USB cable

The checks


battery out


multimeter on beep


+ to -: no beep


a chirp that stops: fine

- Battery OUT. Multimeter on beep.
- Battery connector + to -: NO beep. Board Z, Y, X: +5V to GND: NO beep. A short chirp that stops at once is the capacitors charging - that is fine; a beep that goes on is a short.
- Board X: x8 to x9 (CAN), multimeter on Ω: about 60 Ω. x9 to x5: NO beep.
- Each wheel turns freely by hand and touches nothing.

The USB rule

- The Teensy and the Wi-Fi board also take power from their USB cable. With the battery in at the same time, two supplies meet on the boards.
- So, always: the USB cable OR the battery - never both at the same time.
- To send a program in: battery OUT, then the cable in. Done: cable OUT, then battery in.
- Everything else - the commands, the runs - goes over the Wi-Fi, with the battery in.

Battery in

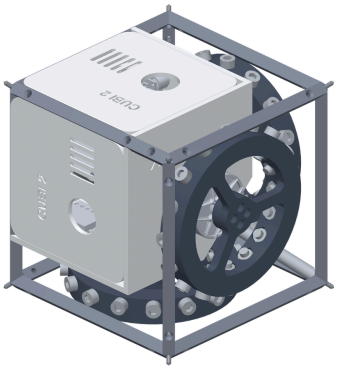
- No USB cable in. Battery in, with your teacher. Watch the cube for 10 s: smoke, a burning smell or a hot part - battery out at once.
- Each motor driver lights its LED. The Teensy and the Wi-Fi board light up.
- Board Y, y1 to y2 (multimeter on V): 4.5 - 5.2 V - the picture below.
- Anything wrong: battery out, tell your teacher.

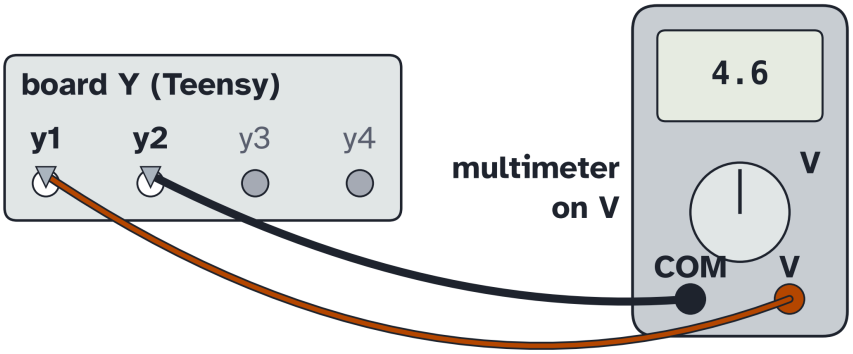
Battery in - the 5 V check

battery IN, with your teacher: does the Teensy get its 5 V?

4.5 - 5.2 V: good

else: battery out, tell your teacher





Cubi 2, battery in

red probe on y1 (+5V), black on y2 (GND)

Check - tick each line. All ticked: a good place to stop.

- ☐ All beep checks right.
- ☐ You can say the rule: USB cable or battery, never both.
- ☐ Battery in: the LEDs light, y1 - y2 4.5 - 5.2 V.

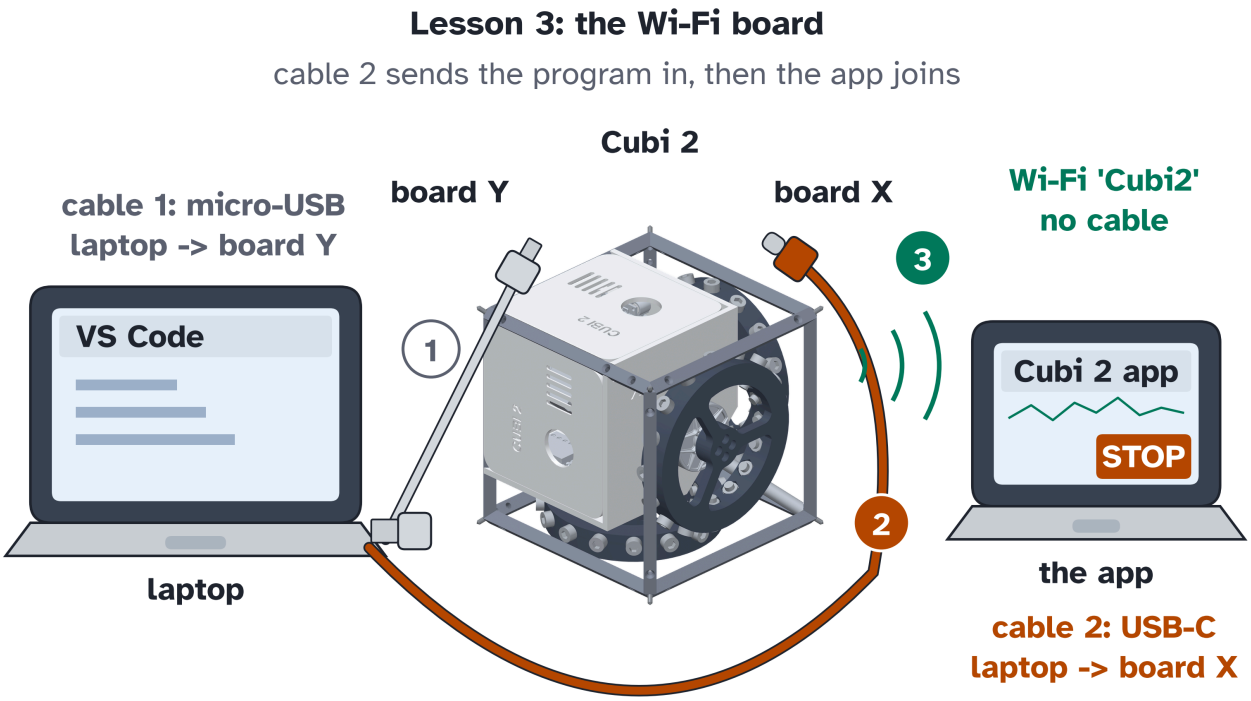
Next: lesson 3, the Wi-Fi board.

Lesson 3 · The Wi-Fi board

Send the program cubi2_radio into the Wi-Fi board on board X. Then open the app.

About 30 minutes. You can stop after any step. You will learn:

- what an upload and a COM port are
- how the laptop talks to the cube without a cable



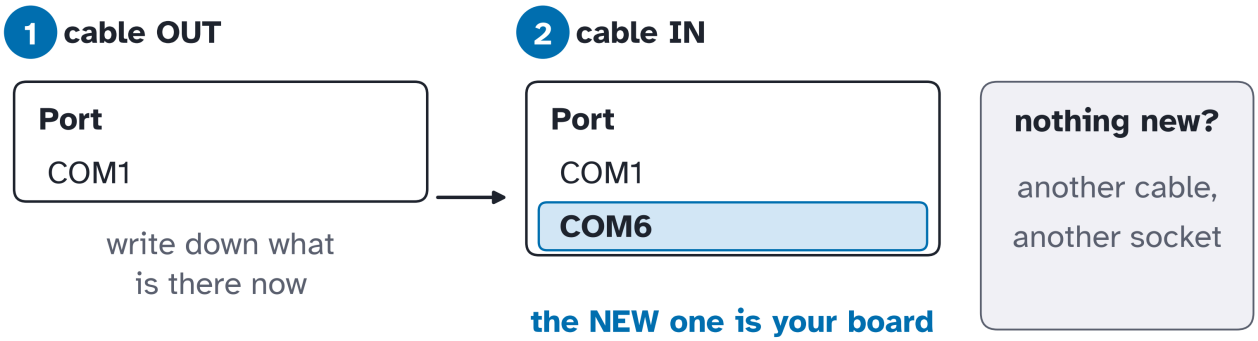
used in this lesson: cable 2 (battery OUT), then Wi-Fi (battery in)

Upload cubi2_radio

- Battery OUT. Cable 2 (USB-C) from the laptop to the Wi-Fi board. ONE cable only: cable 1 (Teensy) OUT - with both in, 'Auto' may pick the Teensy.
- VS Code: File > Open Folder > firmware/cubi2_radio.
- Bottom bar: the arrow (Upload). It builds, then sends. The end: 'Hash of data verified', then SUCCESS. Yellow 'defined but not used' warnings are harmless.

Which port is my board?

Auto picks the wrong one? Look before and after plugging in



The names differ on each system:

Windows COM6 macOS /dev/cu.usbmodem101 Linux /dev/ttyACM0

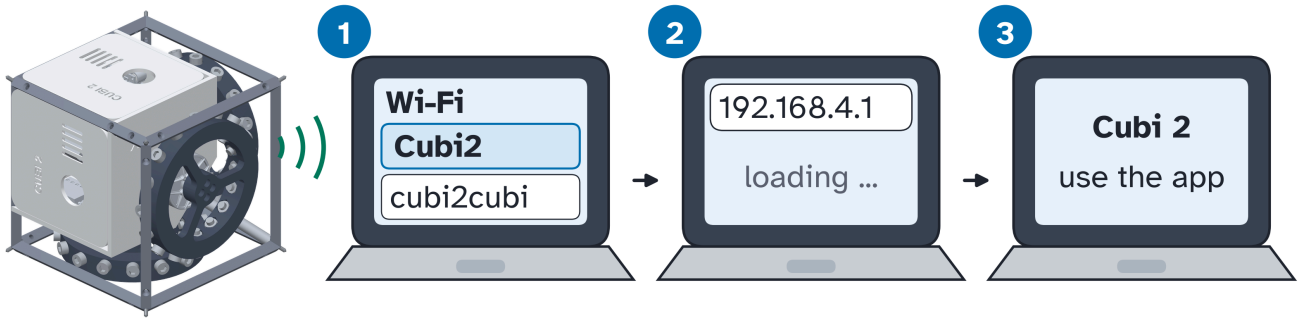
- The board, the USB settings and the speed are in platformio.ini - nothing to choose.
- A port with 'Bluetooth' in its name is never the board.
- Upload fails? If the Wi-Fi board has a BOOT button: hold it while you plug the cable in, then Upload again.

The Wi-Fi

- Cable 2 OUT, then battery in. Laptop: Wi-Fi network 'Cubi2', password 'cubi2cubi'. It may say 'no internet' - stay connected.
- Browser: http://192.168.4.1 - the Wi-Fi board answers in one line: 'Cubi 2 - open the Cubi 2 app'. The rest is in the app.

The laptop joins the cube

battery in - the cube makes its own Wi-Fi



Cubi 2

choose 'Cubi2' password cubi2cubi

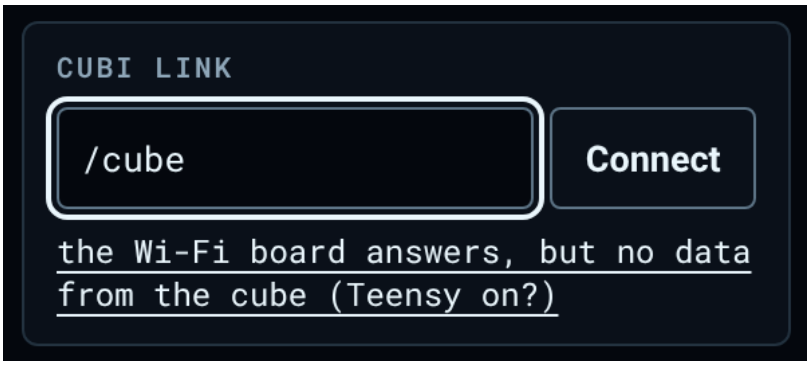
browser: 192.168.4.1

the cube answers

'No internet' is fine: the cube is not the internet.

The app

- Start the app as in lesson 1, 'Node.js and the app': node tools/serve.mjs in Documents/cubi2app, then localhost:8000/html/index.html in the browser. It needs no internet.
- At the top: Cube, then Live Cubi. In the Cubi card type /cube - Connect. The app's server passes it on to the cube at 192.168.4.1.
- No data from the cube yet is right for now: the Teensy has no program yet.
- From lesson 4 on, every command is typed here: Terminal (top right), type, then Send.



Check - tick each line. All ticked: a good place to stop.

- [ ] Upload: SUCCESS.
- [ ] At 192.168.4.1 the cube answers: Cubi 2 - open the Cubi 2 app.
- [ ] The app: Live Cubi, /cube, Connect - the Wi-Fi board answers.

Next: lesson 4, the setup program.

Lesson 4 · The drivers' names

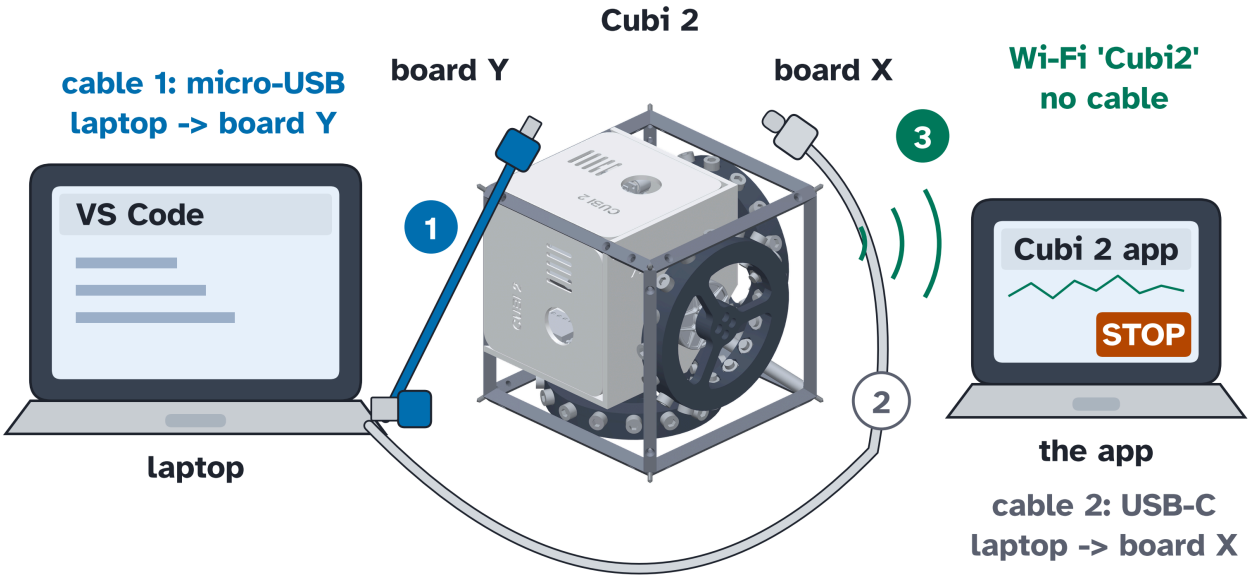
Send cubi2_setup into the Teensy and give each motor driver its own number.

About 40 minutes. You can stop after any step. You will learn:

- what the CAN bus is and why each driver needs its own id
- how to talk to the Teensy with commands

Lessons 4 to 7: the Teensy

cable 1 sends the program in, then the app talks to it



used in these lessons: cable 1 (battery OUT), then Wi-Fi (battery in)

Upload cubi2_setup

- Battery OUT. Cable 1 (micro-USB) into the Teensy, cable 2 out.
- File > Open Folder > firmware/cubi2_setup. Upload (the arrow). A small Teensy window may open and close - normal. It asks for the button on the Teensy? Press it once.
- The Teensy's upload shows no hash: its first lines in the app are the proof.
- Upload done: cable 1 OUT, then battery IN. Wait 10 s - the motor drivers start.
- The app: Live Cubi, Connect, Terminal. Type help, then Send.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
  === Cubi 2 - SETUP ===
    (no saved settings yet - the model's values are used)
    OK   IMU at 0x68
  Commands: scan | find | ...  (the list, as below)
› help
  Commands: scan | find | id OLD NEW | cal ID | limits | spin ID
AMPS | enc ID | ical ID [A] | imu | i2c | bias | face x|y|z |
           wheels | corner 0|7 | battery | link | show | save |
defaults
```

Who answers

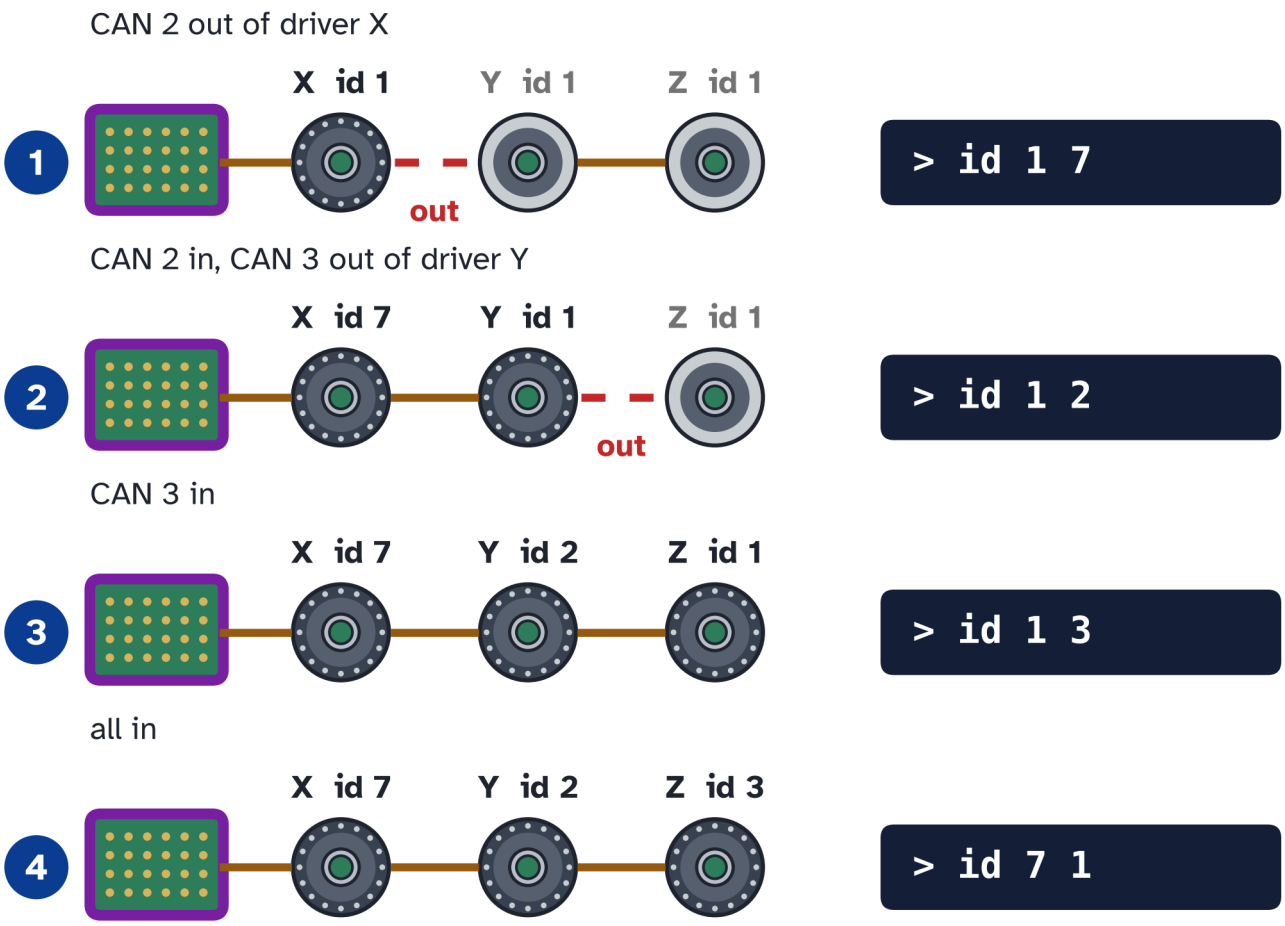
- The CAN bus is one cable that visits all 3 drivers. Each needs its own number (id).
- New drivers all have id 1, so the first scan may show one driver, or a mess. Normal.
- No answer at all: 10 s after the battery went in? CAN plugs clicked in? Termination plug in driver Z, port 13? 'find' searches every id (about 10 s).

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› scan
  Scanning CAN ids 1 ... 8
    id 1  firmware 3.0.0  hash ...  24.1 V  IDLE  NOT calibrated
errors 0/0
    1 driver(s) found
```

Each driver its own id

- Goal: driver X = id 1, driver Y = id 2, driver Z = id 3. New drivers all have id 1, so they get their numbers ONE AT A TIME: the others are cut off the CAN cable meanwhile.
- The CAN cable: board X > driver X > driver Y > driver Z > the end plug (port 13). 'X-Y' is the short cable between driver X and driver Y; 'Y-Z' the one between Y and Z.
- ALWAYS battery OUT to pull or push a CAN plug. Battery IN, wait 10 s, Connect in the app again (the Wi-Fi comes back with the battery), then the commands.
- Best done before the wheels go on: then the CAN plugs are easy to reach.



Row 1 of the picture - only driver X on the cable. Battery OUT, unplug X-Y (the cable CAN 2) at driver X. Battery IN, then:

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› scan
    id 1  firmware 3.0.0 ... 24.1 V  IDLE  NOT calibrated
errors 0/0
› id 1 7
    the driver restarts - wait 10 s ...
    OK   the driver now answers on its new id (saved)
```

Driver X is now id 7 - only for a moment, to keep id 1 free. Row 2: battery OUT, plug X-Y back, unplug Y-Z (the cable CAN 3) at driver Y. Battery IN, then:

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› scan
    id 1  ...
    id 7  ...
› id 1 2
    the driver restarts - wait 10 s ...
    OK   the driver now answers on its new id (saved)
```

Row 3: battery OUT, plug Y-Z back (all plugs in now). Battery IN, then:

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› scan
    id 1  ...
    id 2  ...
    id 7  ...
› id 1 3
    the driver restarts - wait 10 s ...
    OK   the driver now answers on its new id (saved)
```

Row 4: driver X back to id 1, and the check:

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› id 7 1
    the driver restarts - wait 10 s ...
    OK   the driver now answers on its new id (saved)
› scan
  Scanning CAN ids 1 ... 8
    id 1  firmware 3.0.0 ... 24.1 V  IDLE  NOT calibrated
errors 0/0  = driver X
    id 2  firmware 3.0.0 ... 24.1 V  IDLE  NOT calibrated
errors 0/0  = driver Y
    id 3  firmware 3.0.0 ... 24.1 V  IDLE  NOT calibrated
errors 0/0  = driver Z
    3 driver(s) found
```



Two drivers on one id?

A scan with two drivers on the same id, or a mess: a plug was still in - battery OUT, check the row, again.



Check - tick each line. All ticked: a good place to stop.

- ☐ The upload worked; 'help' answers in the app.
- ☐ scan shows 3 drivers: id 1 = X, id 2 = Y, id 3 = Z.

Next: lesson 5, calibration.

Lesson 5 · Teach the drivers their motors

Calibration, safe limits, a first gentle spin.

About 35 minutes. You can stop after any step. You will learn:

- what calibration is: the driver measures its motor and its angle sensor
- why limits keep the cube safe

Calibrate

- The cube lies still on a face. Hands off: the wheel will turn.
- cal 1, then cal 2, then cal 3. Each 20-40 s: a beep, the wheel turns slowly one way, then back.
- The numbers below are an example; the 3 drivers show about the same. 'calibrated and saved in the driver' - done for good.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› cal 1
    Calibrating: the wheel beeps and turns slowly one way, then
back. Hands off. 20-40 s.
    ..........
    R 0.400 ohm  L 150.0 uH  pole pairs 11  motor errors 0
sensor errors 0
    OK   calibrated and saved in the driver
› cal 2
    ...  OK   calibrated and saved in the driver
› cal 3
    ...  OK   calibrated and saved in the driver
```

Limits and a gentle spin

- limits: at most 8 A per motor and 4297 rpm - kept inside the drivers.
- spin 1 1.0 - wheel X speeds up for 2 s (1.0 A). Then spin 2 1.0, spin 3 1.0.
- Less than 1.0 A may not move the heavy wheel - that is normal. A wheel that rattles, rubs or stays at 0 rpm with 1.0 A: stop, tell your teacher.
- A new motor driver later? Then cal and limits again for it: a new driver comes with its own lower limits.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› limits
    driver X (id 1): current limit 8.0 A, speed limit 4297 rpm
    driver Y (id 2): current limit 8.0 A, speed limit 4297 rpm
    driver Z (id 3): current limit 8.0 A, speed limit 4297 rpm
› spin 1 1.0
    5 rpm   current 1.01 A (asked 1.00 A)   RUNNING
    164 rpm   current 1.09 A (asked 1.00 A)   RUNNING
    ...
    641 rpm   current 0.97 A (asked 1.00 A)   RUNNING
    errors: system 0  controller 0  motor 0  sensor 0
    OK   stopped (the wheel coasts down)
› spin 2 1.0
    ...  OK   stopped (the wheel coasts down)
› spin 3 1.0
    ...  OK   stopped (the wheel coasts down)
```

Check - tick each line. All ticked: a good place to stop.

- ☐ cal 1, cal 2, cal 3: 'calibrated and saved'.
- ☐ limits: 3 lines, 8.0 A each.
- ☐ spin: each wheel turns smoothly.

Next: lesson 6, the IMU and the wheel test.

Lesson 6 · Which way is up

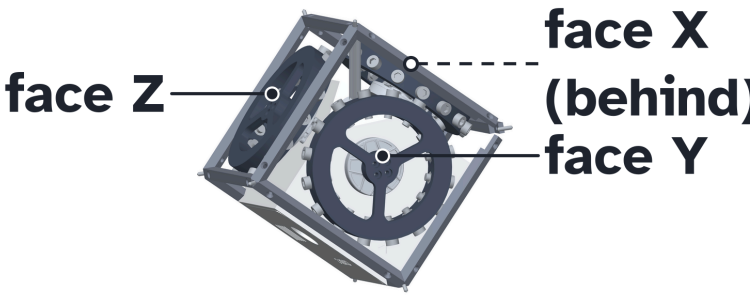
The IMU, then the wheel test. Then save.

About 40 minutes. You can stop after any step. You will learn:

- what a gyroscope and an accelerometer measure
- how the cube finds out which wheel is which

The IMU

- The IMU measures turning (gyroscope) and gravity (accelerometer). It tells 'up'.
- bias - the cube still on the table for 3 s: the gyroscope's zero.
- face x - FIRST turn the cube onto face X (wheel X at the bottom), THEN type face x; still for 2 s. Then the same for face y and face z. After the 3 faces: 'orientation measured'; each face under 5 deg from the model. In doubt? bias and all 3 faces again, then save.



- The commands of 'The IMU', in order - the cube still on the table, then on each face:

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

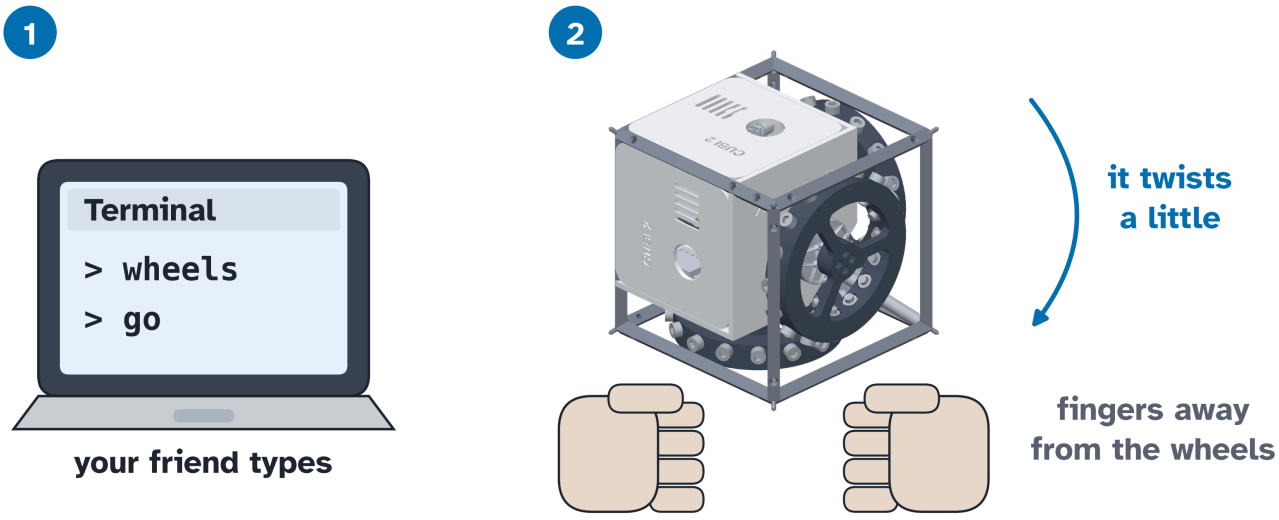
```
› bias
    Keep the cube STILL for 3 s ...
    gyro zero 0.112 -0.054 0.031 deg/s
    OK   done - 'save' to keep it
› face x
    The cube lies on face X (wheel X at the bottom). Still for 2
s ...
    up in the IMU frame: -0.815 +0.409 +0.411
    the model expects this face 1.2 deg away
    now the other faces (face x, face y, face z)
› face y
    ...  the model expects this face 0.9 deg away
› face z
    ...  the model expects this face 1.4 deg away
    OK   orientation measured - 'save' to keep it
```

The wheel test

- Type wheels. A friend has the laptop; you hold the cube IN THE AIR from below, on open hands, loosely - not gripped. Fingers away from the wheels, out of the cage. Your friend sends 'go'.
- Each wheel kicks for 0.3 s; the cube twists the other way. That is how the Teensy learns which wheel is which and which way it pushes. 'not clear': hold looser, repeat.
- Never skip it: the brain does not start without it.

The wheel test - two people

the cube in the air: each wheel kicks, the cube twists the other way



Not clear? Hold looser and repeat. Never skip it - the brain needs it.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› wheels
    Hold the cube IN THE AIR with both hands, loosely, wheels
free.
    Each wheel kicks for a moment; the cube twists the other
way. Send 'go' to start.
› go
    driver X: wheel 750 rpm, cube twist -0.041 +0.002 -0.003 rad
      -> driver X turns the wheel on axis x, same way (+)
    ...
    OK   wheel test done - 'save' to keep it
```

Save

- link - the Wi-Fi board must answer. show - everything at once.
- save - into the Teensy. It is kept with the power off.
- Without 'save' the Teensy forgets lesson 6 at the next power-up. The drivers keep their ids and calibration (lessons 4 and 5) themselves.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› link
    OK   the Wi-Fi board answers
› show
    drivers X Y Z: ids 1 2 3 | wheel axes x y z | signs +1 +1 +1
| wheel test done
    ...
› save
    OK   saved
```

Check - tick each line. All ticked: a good place to stop.

- ☐ bias, face x, face y, face z: 'orientation measured'.
- ☐ wheels: 'wheel test done'.
- ☐ save: 'saved'.

Next: lesson 7, the brain and the balance point.

Lesson 7 · The brain and its balance point

Send cubi2_brain into the Teensy. Cubi checks itself, then finds its own balance point by hanging.

About 40 minutes. You can stop after any step. You will learn:

- how the cube checks its own sensors
- why hanging shows the true balance point
- Cubi always asks first: you answer Yes or No

Upload cubi2_brain

- Battery OUT, cable 1 into the Teensy. File > Open Folder > firmware/cubi2_brain. Upload.
- Cable 1 OUT, battery IN. From now on the brain starts by itself every time the battery goes in.
- The app: Live Cubi, Connect, Terminal. The cube lies still on a face.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
  === Cubi 2 - BRAIN ===
  IMU ok
  waiting 10 s for the motor drivers to start ...
  3 drivers ok
  self-check - keep the cube still for 2 s ...
  check: gyro zero ok | |g| 1.002 ok | noise 0.05 deg/s ok |
vibration 3 mg ok | drivers ok
    all fine
  type 'go corner' (or 'help')
```

The self-check

- At every start Cubi checks itself for 2 s: keep it still.
- It checks the IMU (its zero, gravity, its noise), how much the table shakes and the 3 drivers. check does it again at any time - then with the battery too.
- 'all fine': go on. A line with '!!', or a question: read it and answer it (see 'Cubi always asks first').
- drivers: the 3 drivers - found, their state, their errors. Nothing moves.

Cubi always asks first

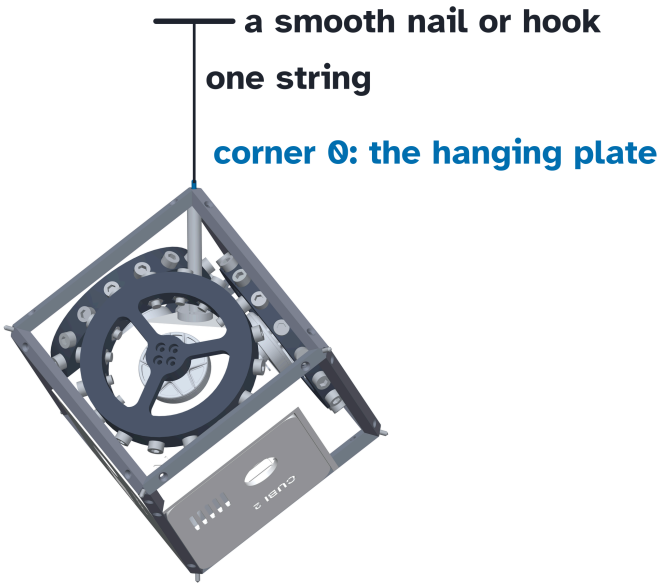
- Cubi changes or saves nothing by itself. It asks, and you answer.
- In the app: the box 'The cube asks', with two buttons that say what you saw. Choose the one that is true. 'Cubi's own words' shows its exact line.
- In the Terminal: a line that starts with '?'. Type yes or no, then Send.
- Questions come one at a time. A new run drops the open ones; after it Cubi asks again what still matters.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
  ? the gyro zero has moved 0.060 -0.012 0.008 deg/s since it
was measured - take the new zero and save? (yes / no)
› yes
  new gyro zero taken and saved
```

The balance point, by hanging

- Every Cubi is a little different: its cables, its covers, its battery. So it finds ITS OWN balance point.
- Take the pointed screw out of the tip of corner 0. Screw the hanging plate in its place.
- One string through both rings of the plate, over a smooth nail or hook. Cubi hangs from corner 0, the wheels off.
- Wait until it hangs still - turning slowly about the string is fine. Then type zero hang.
- 'keep it and save?' - yes. Then the plate out and the pointed screw back in.
- Do it again whenever something changes: the covers, the cables, the battery.



The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› zero hang
  corner 0 point set by hanging: up +0.5839 +0.5752 +0.5729 (
moved 0.78 deg)
  ? corner 0 point set by hanging (moved 0.78 deg) - keep it and
save? (yes / no)
› yes
  corner 0 point saved
```

'it still swings … wait until it hangs still': wait a little, then zero hang again.

Is the floor flat?

- Lay Cubi on a face on the mat where it will balance. Type slope.
- Turn it half a turn on the same spot, the same face down. Type slope turn.
- Under 1 deg is fine. More: a flatter place.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› slope
  lying on face -x: tilted 0.74 deg (the floor and the IMU
together). Turn it half a turn on the same spot, then 'slope
turn'
› slope turn
  the floor slopes 0.43 deg | the IMU reads 0.74 deg off on face
-x (its own error, it stays)
```

Check - tick each line. All ticked: a good place to stop.

- ☐ 'IMU ok', '3 drivers ok', the self-check: 'all fine'.
- ☐ zero hang: 'keep it and save?' - yes: 'corner 0 point saved'.
- ☐ The pointed screw is back in corner 0.
- ☐ slope, slope turn: the floor slopes under 1 deg.

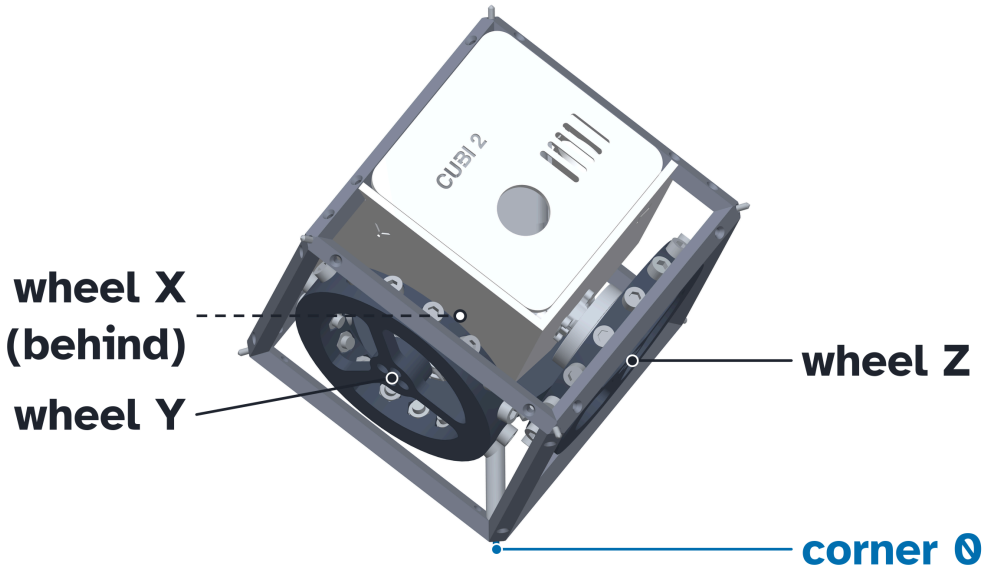
Next: lesson 8, on the corner.

Lesson 8 · On the corner

All three wheels together: Cubi stands on corner 0. After each run it tells you how it went.

About 45 minutes. You can stop after any step. You will learn:

- how a feedback controller works: measure, correct, 250 times a second
- how to read the report after a run
- why you change one thing at a time



Ready, lift, let go

- Type go corner (or press Start in the app) - 'ready'.
- Corner 0 on the mat, fingertips on the top corner. Bring it upright slowly. Within 4 deg of its balance point it takes over: 'BALANCING'. Let go gently.
- It stops by itself beyond 20 deg, a wheel above 4100 rpm, a silent driver, a low battery.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› go corner
  ready: corner. Hold the cube on its corner, slowly bring it
upright - it takes over within 4 deg.
  BALANCING
```

Stop, then the report

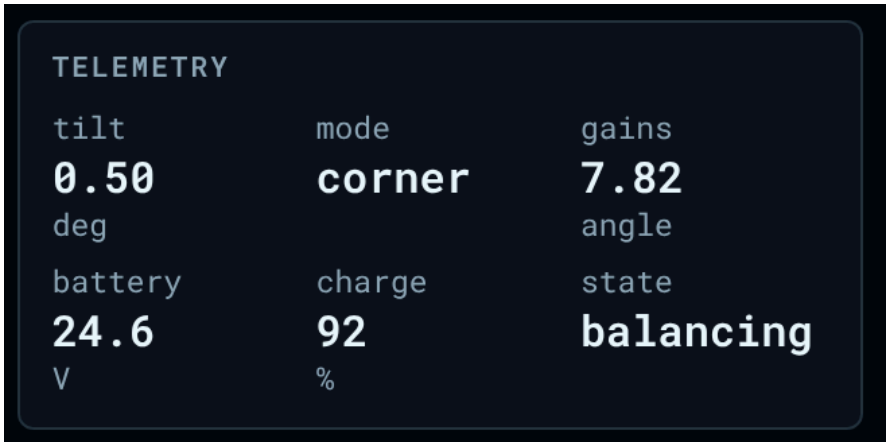
- To end: type stop (or press Stop in the app) WITHOUT touching Cubi, then catch it. A touch while it balances makes a wheel run up to its limit.
- Then Cubi says how the run went: how much it leaned (tilt), how fast it moved (rate), the wheels at their limit, and the shaking - slow, middle, fast.
- 'disturbances': a knock (jolt) or a single wrong reading (spike). From a run with a knock in its last 4 s it learns nothing.
- Under 1.5 s of balancing the run is too short for the coach - it says so.
- SAFETY: stop brakes the wheels in a few seconds; then 'wheels stopped'. They do not slow down: battery OUT. Fast wheels keep the boards on for a while - do not touch them until they stop.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› stop
  off - wheels 120 -85 40 rpm
  how it went: tilt 0.31 deg | rate 0.9 deg/s | wheels at the
limit 0 % | slow 71 % (1.2 Hz) middle 21 % (6.0 Hz) fast 8 % (28
Hz)
  disturbances: none | vibration 3 mg
    calm 0.36 - nothing to change
```

The balance point after a run

- While it balances, Cubi learns where it really balances. Moved 0.5 deg or more: it asks 'keep it and save?'. Was the run calm, and did nobody touch it? Then yes.
- Under 0.5 deg: 'the balance point is good' - nothing to change.
- No learned point? The wheels' average push shows where the point is: Cubi asks 'move it there and save?'. By hand it is the same: fix, then save.
- Go again, until the point moves less than 0.3 deg. Why again? Over a wrong point the cube leans to the other side to stay up - so each run shows the point a little better.



The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
  ? it learned a new balance point while it balanced (corner
0.94 deg) - keep it and save? (yes / no)
› yes
  kept and saved
```

Or, after a run with no learned point:

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
  ? the wheels pushed 41 mNm one way on average: the true corner
point looks 0.62 deg away - move it there and save? (yes / no)
› yes
  moved and saved - 'go' again
```

The coach: one change at a time



It asks first

After a run the coach suggests at most ONE change - and asks first.



What it saw

A fast shake the wheels follow: a filter that calms it. A twist on the tip: less twist. A wobble a few times a second: a calmer reaction. A slow, big sway: a stronger reaction.



Keep it or go back

Yes: Cubi uses it and saves nothing yet. After the next run it compares - better or about the same: 'keep it and save?'; worse: 'go back?'.



A long, calm run

1 minute or more: type show and write its settings down - a good start for next time.



Check - tick each line. All ticked: a good place to stop.

- ☐ Cubi balances on corner 0 for 30 s.
- ☐ stop without touching it; the report comes.
- ☐ The balance point questions answered; fix moves it less than 0.3 deg.
- ☐ save: 'saved'.

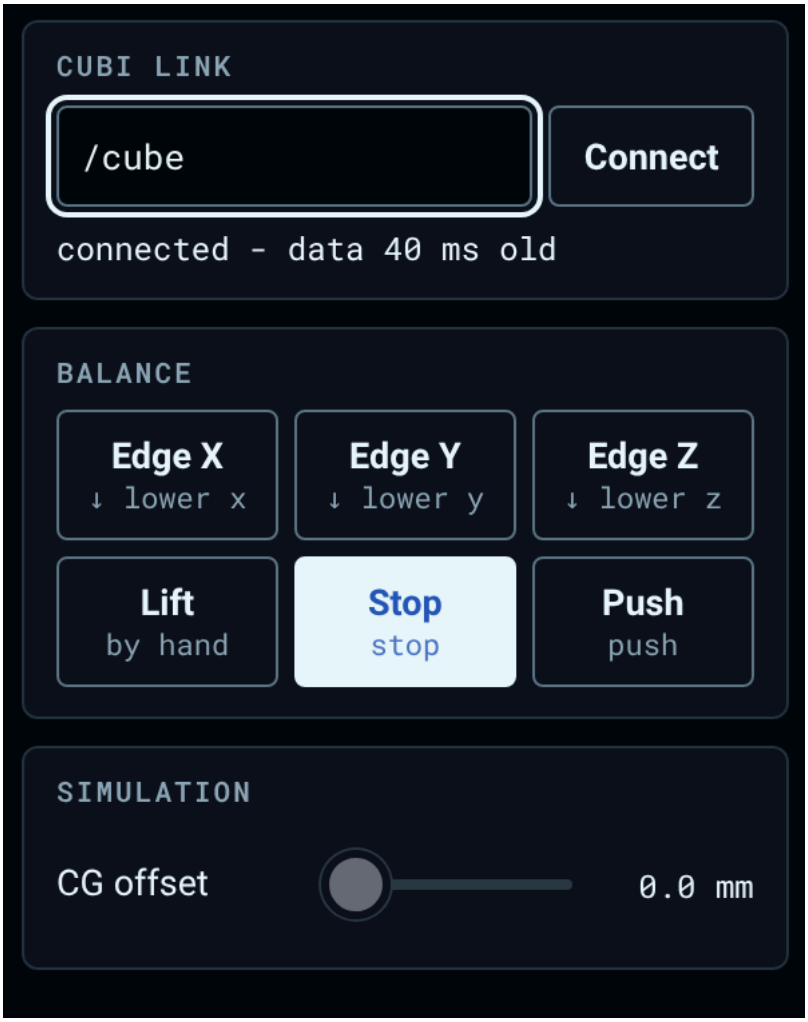
Next: the Cubi 2 app.

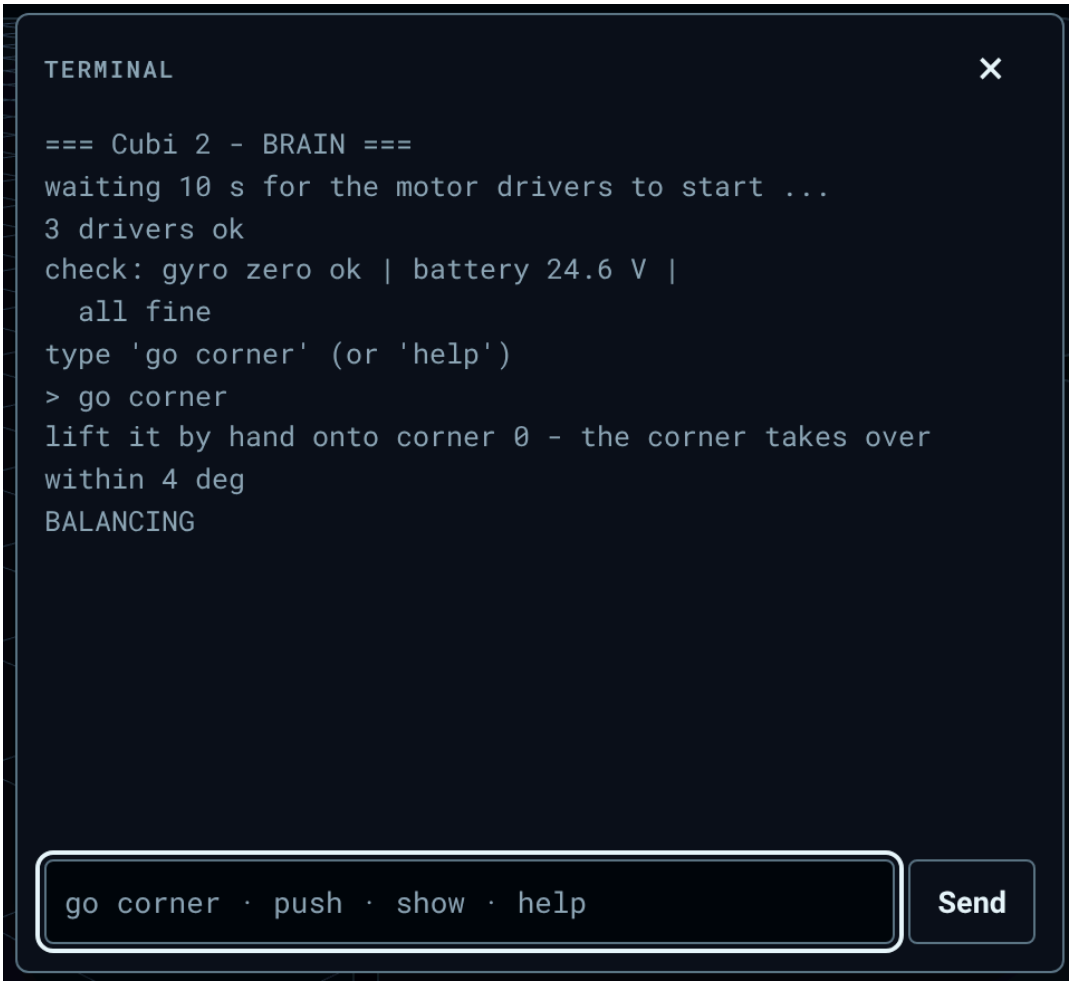
The Cubi 2 app

Wi-Fi 'Cubi2', password 'cubi2cubi'. The app: Cube, Live Cubi, /cube in the Cubi link card, Connect.



- Top bar: Sphere or Cube - which Cubi. Simulation or Live Cubi - the model on the laptop, or the real cube.
- Balance: Edge X / Y / Z while Cubi balances on the corner: it lowers itself onto that edge and drops the last part (about 1 cm). Keep the mat under it. Stop: the wheels off.
- Lift and Push are for the simulation. On the real Cubi they show what to do by hand.
- Telemetry: tilt, mode, gains, battery (V and %), state. Wheels: the last 20 s.
- Runs: the charts of the last run - tilt and wheels.
- Settings: Zero, Fix, Save, Where, Swing, CG, Show, Status, Help - each sends its command; the answer is in the Terminal.
- Terminal: Cubi's messages; type any command, then Send.


CUBI LINK
/cube
Connect
connected - data 40 ms old
BALANCE
Edge X
↓ lower x
Edge Y
↓ lower y
Edge Z
↓ lower z
Lift
by hand
Stop
stop
Push
push
SIMULATION
CG offset
0.0 mm



New programs

- A new program goes in by the USB cable, as in lessons 3 and 4: battery OUT, cable in, Upload. Done: cable OUT, then battery in.

Settings by hand - for your teacher

The coach suggests them; these are the same by hand. One change at a time, then a run, then save.

The Cubi 2 app - Live Cubi > Terminal: type, then Send — type, then Enter

```
› shake 25 0.7
  shake filter 25 Hz, width 0.7
› show
  OFF corner | tilt 0.40 deg | battery 24.1 V 80 % | wheels 0 0
0 rpm | ...
    rate filter 15 Hz | shake filter 25 Hz width 0.7 | twist
0.30 | spin 1.00 | learn 0.20 /s | takes over within: corner
4.0, edges 4.0 deg
    stops beyond 20.0 deg (fall) | above 4100 rpm (wheelmax)
› save
  saved
```

save keeps all of these, also with the power off. Start values - corner: 7.82, 0.890, 0.000356; edge: 7.13, 0.775, 0.000352.

- **k ANGLE RATE WHEEL**
  the gains of the mode of the last 'go' (go edge z, stop - then k for the edge)
- **shake HZ [W]**
  the shake filter: 25 Hz, width 0.7 at power-up; shake 0 = off
- **filter HZ**
  the rate filter, 15 Hz
- **twist F · spin F**
  how firmly the tip is held against twisting · the wheels slowed along the diagonal
- **learn L**
  how fast it learns its point while it balances; learn 0 for a clean 'fix'

start DEG · fall DEG · wheelmax RPM
take over within (4 - 8 deg) · stop beyond · the wheels' limit

up X Y Z
a balance point that worked, typed back ('show' prints it) - write it down after a calm run

dump · dump all
the last 4 s of a run, line by line - to find what went wrong

When something is wrong

First: copy the message into a search engine or ask an AI assistant. These are the answers for Cubi 2 itself.



No port, or 'Auto' picks the wrong one

a DATA cable? another USB socket? Click 'Auto' in the bottom bar - a new port must appear when you plug in. Linux: the USB rights (lesson 1, 'The USB rights')? macOS: 'Allow accessory'?



Build FAILED

the first Build needs internet (it downloads the tools). Never change the .h files or platformio.ini. Always open the program folder itself (the one with platformio.ini).



Build SUCCESS is not an upload

press Upload too. A file of a program changed? Replace the whole folder - the program needs all its files.



The Teensy's red LED stays on

no program runs - an upload did not finish. Upload again.



No driver answers, or 'drivers not found'

battery OUT, battery IN, wait 10 s, then again. Still nothing: CAN plugs clicked in? termination plug in driver Z, port 13? On board X: yellow CANH x8, black CANL x9.



'did not start ... controller'

The driver stopped on its current - 'go' again after 10 s, or battery out and in.



Driver firmware UNKNOWN

the programs know driver firmware 1.3 to 3.2. Tell your teacher.



Calibration fails

wheel blocked? magnet centered (Magnets guide)? Battery above 22 V? enc ID: turn the wheel by hand - the angle must follow. ical ID: the calibration current.



A driver stuck at the start (in its bootloader)
setup and drivers restart it by themselves - wait 10 s. (Teacher: only drivers with firmware 3.0 did this.)



IMU not found
the IMU plug; poles y4 SDA, y5 SCL, y6 3V3, y7 GND. i2c shows the bus.



'digital spikes'
Or 'IMU noisy': a loose wire on the IMU's cable - check its plug.



'faces x, y, z do not fit together'
The cube moved - repeat all three.



'two drivers on one axis'
The ids are mixed up - scan, then the id steps again.



'a driver went silent'
A CAN plug half out; the battery very low.



'battery low'
Or 'charge it': your teacher charges it (it stops below 21.0 V).



'unknown command'
(setup) or 'unknown - help' (brain): a stray character - type it again.



'drivers not found'
In the brain: battery out and in, wait 10 s, then drivers.



A run went wrong and you want to see why
stop, then dump - the last 4 s, for your teacher.



The app
'no connection': on the Wi-Fi 'Cubi2'? 'no data from the cube': the battery in? the brain running? the link wires (BRX / BTX)? 'CORS … missing' in the browser's console: the Wi-Fi board has an old program - lesson 3 again.



STOP does not slow the wheels

battery OUT. Fast wheels keep the boards on for a while (they work like generators) - do not touch the wheels until they stop.

All the commands

cubi2_setup

```
› scan          > find          > id OLD NEW     > cal
ID
› limits        > spin ID AMPS  > enc ID          > ical
ID [A]
› imu           > i2c           > bias           > face
x | y | z
› wheels        > corner 0 | 7  > battery        > link
› show          > save          > defaults
```

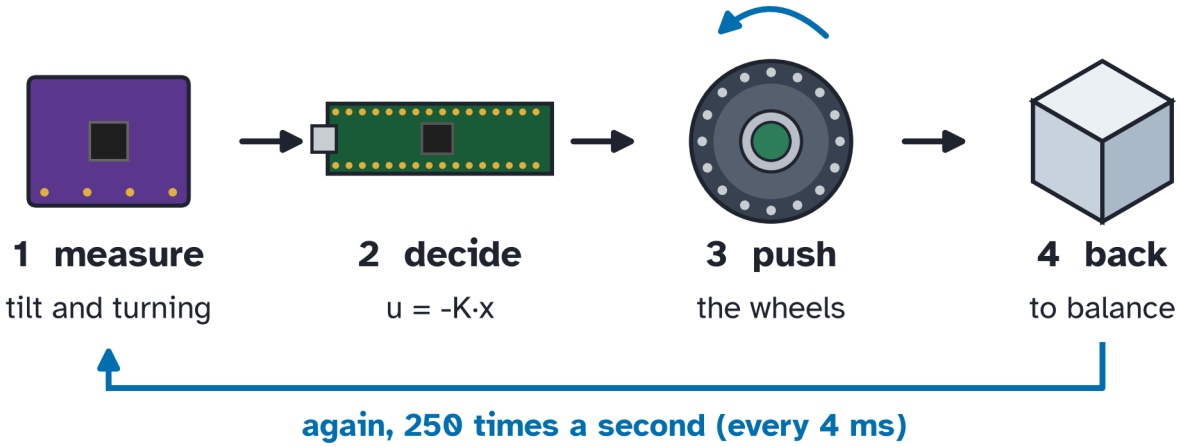
cubi2_brain - balance

```
› go corner     > go edge x | y | z               >
stop
› yes | no      > check         > slope | slope turn
› zero hang     > fix           > up X Y Z       > save
› coach on | off                > profile        >
drivers
› status        > show          > help           > tele
on | off
```

cubi2_brain - by hand

```
› k ANGLE RATE WHEEL            > shake HZ [W]   >
filter HZ
› twist F       > spin F        > learn L        >
start DEG
› fall DEG      > wheelmax RPM  > dump | dump all
```

How it balances



- Feedback: measure the tilt, push against it, measure again - 250 times a second.
- Each wheel's push = angle gain x tilt + rate gain x turning speed + wheel gain x wheel speed.
- The gains come from LQR, a method that finds the best gains for a model of the cube: 1.92 kg, center of gravity 124 mm above corner 0.
- Two filters calm the push: the rate filter (15 Hz) and the shake filter (25 Hz), for the fast shaking on hard floors.
- The balance point: from hanging, then learned while it balances - kept only when you say yes.

Words



Editor / extension
where you write code / an add-on that teaches the editor something new



Build
turning the program text into machine code for one kind of small computer



Upload
sending the built program from the laptop into the small computer



Port
the name the system gives a USB connection: COM5 (Windows), /dev/cu.usbmodem101 (macOS), /dev/ttyACM0 (Linux)



PATH
the list of folders where the system finds programs typed in a terminal



Library
ready-made code, e.g. the one that reads the IMU



CAN bus, id
one cable for all drivers; each has its own number



Calibration
the driver measures its motor and finds where its magnet points



IMU
gyroscope (turning) + accelerometer (gravity): it tells 'up'



Gain
how strongly the brain reacts to an error



Balance point
the angle where the cube would stand by itself



Take-over angle

how close to its balance point Cubi starts balancing: 4 deg



Coach

the part of the brain that reports each run and suggests one change

PATH - only if you want it

PATH is the list of folders where your system looks for a program you type in a terminal. This guide does not need it: VS Code and PlatformIO find their own tools, and the PlatformIO terminal (PlatformIO icon > Quick Access > New Terminal) already knows 'pio'.

The 'code' command in any terminal

- Windows: the installer's 'Add to PATH' does it - log out and in once. Not there? Start, type the first line, open 'Edit environment variables for your account' > Path > Edit > New, type the second line > OK, OK.

Windows - Start: the search box — type, then Enter

```
› environment variables
```

Windows - Edit environment variable > New — type, then Enter

```
› %LOCALAPPDATA%\Programs\Microsoft VS Code\bin
```

- macOS: in VS Code press Cmd+Shift+P, type the line below, Enter. Linux: the .rpm package already does it.

VS Code - Cmd+Shift+P (macOS): the command box — type, then Enter

```
› Shell Command: Install 'code' command in PATH
```

The 'pio' command in any terminal

- Windows: the same way as for 'code', with this folder:

Windows - Edit environment variable > New — type, then Enter

```
› %USERPROFILE%\.platformio\penv\Scripts
```

- macOS: add the line below to the file ~/.zshrc; Linux: to ~/.bashrc. Then open a new terminal.

Text editor - the last line of ~/.zshrc (macOS) or ~/.bashrc (Linux), then save

```
› export PATH=$PATH:$HOME/.platformio/penv/bin
```

Check it

- Open a NEW terminal (an old one keeps the old PATH). Type the command - a version number (yours differs from the example) means PATH is right. 'not found' / 'not recognized': the folder is wrong or the terminal is old.

Terminal - Windows: PowerShell · macOS and Linux: Terminal — type, then Enter

```
› code --version
  1.105.1
› pio --version
  PlatformIO Core, version 6.1.18
```